

AdaGCL+: An Adaptive Subgraph Contrastive Learning Toward Tackling Topological Bias

Yili Wang , Yaohua Liu, Ninghao Liu , *Member, IEEE*, Rui Miao, Ying Wang , and Xin Wang 

Abstract—Large-scale graph data poses a training scalability challenge, which is generally treated by employing batch sampling methods to divide the graph into smaller subgraphs and train them in batches. However, such an approach introduces a topological bias in the local batches compared with the complete graph structure, missing either node features or edges. This topological bias is empirically shown to affect the generalization capabilities of graph neural networks (GNNs). To address this issue, we propose adaptive subgraph contrastive learning (AdaGCL) that bridges the gap between large-scale batch sampling and its generalization poorness. Specifically, AdaGCL augments graphs depending on the sampled batches and leverages a subgraph-granularity contrastive loss to learn the node embeddings invariant among the augmented imperfect graphs. To optimize the augmentation strategy for each downstream application, we introduce a node-centric information bottleneck (Node-IB) to control the trade-off regarding the similarity and diversity between the original and augmented graphs. This enhanced version of AdaGCL referred to as AdaGCL+, automates the graph augmentation process by dynamically adjusting graph perturbation parameters (e.g., edge dropping rate) to minimize the downstream loss. Extensive experimental results showcase the scalability of AdaGCL+ to graphs with millions of nodes using batch sampling methods. AdaGCL+ consistently outperforms existing methods on numerous benchmark datasets in terms of node classification accuracy and runtime efficiency.

Index Terms—Large-scale graph data, graph neural networks, graph contrastive learning, graph data augmentation.

I. INTRODUCTION

GRAPH neural networks (GNNs) have gained significant popularity in processing graph-structured data [1], [2], [3], [4]. Particularly, GNNs utilize a message-passing strategy,

where node embeddings are learned by recursively aggregating information from their direct neighbors [5]. The vanilla training of GNNs requires taking the entire graph as input and back-propagating the loss to update model weights [6], [7], [8], [9]. This process imposes unaffordable computational demands to store all the node embeddings, which restricts their scalability when applied to large-scale graphs [10], [11]. Consequently, efficiently training GNNs on large-scale graphs while maintaining high generalization performance remains a significant and ongoing challenge. Several methods have been proposed in recent years to address the scalability issue of training GNNs [12], [13], [14]. These methods rely on batch sampling and involve dividing the input graph into subgraphs (e.g., by using clustering). GNNs are then trained with a batch of subgraphs, which significantly reduces the amount of data and required storage. However, the batch training approach fails to preserve the complete neighborhood structure of nodes since the edges among subgraphs are discarded. As a result, each local batch unavoidably contains topological bias, which refers to the distortion of the graph's structural information within the sampled subgraphs. This bias can influence the node embeddings and lead to inaccurate representation learning. As demonstrated by an empirical study in Fig. 1(a), the topological bias quantified by average node degree is amplified as the batch size decreases since more connectivities are discarded. Moreover, according to Fig. 1(b), the topological bias degrades the node embedding learning and leads to a larger performance gap between training and testing. We provide a detailed explanation of the experimental setup and each subfigure of Fig. 1 in Section III-A. Compared with the training phase interfered with by the topological bias, GNNs are often tested by computing the node embeddings on the full graph associated with exact structure information.

The problem of topological bias motivates us to leverage contrastive learning [15], [16], [17]. In general, contrastive learning introduces noise to obtain a number of data views from the input graph and generates robust node embeddings invariant to graph perturbations. The key idea is to encourage the representations of positive pairs $(x, x+)$ to be close, while those of negative pairs $(x, x-)$ are far away from each other [18]. The topological bias among the randomly augmented graphs is thus mitigated. Recently, significant advances have been made in contrastive learning for graph analysis [19]. Despite the effectiveness, most of them only focus on processing small graphs e.g., graphs each with tens of nodes in graph-level classification problems [20], or a graph with thousands of nodes in node-level tasks [21].

Received 27 November 2023; revised 27 April 2025; accepted 19 May 2025. Date of publication 5 June 2025; date of current version 6 August 2025. This work was supported in part by the National Science and Technology Major Project under Grant 2021ZD0112500, in part by the National Natural Science Foundation of China under Grant 62372211 and Grant 62272191, in part by the Science and Technology Development Program of Jilin Province under Grant 20250102216JC, and in part by the Natural Science Foundation of Jiangsu Province under Grant BK20210642. Recommended for acceptance by Z. Wei. (Corresponding author: Xin Wang.)

Yili Wang, Rui Miao, and Xin Wang are with the School of Artificial Intelligence, Jilin University, Changchun 130012, China (e-mail: wangyl21@mails.jlu.edu.cn; ruimiao20@mails.jlu.edu.cn; xinwang@jlu.edu.cn).

Yaohua Liu is with the School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: yaoh.liu@nuist.edu.cn).

Ninghao Liu is with the University of Georgia, Athens, GA 30602 USA (e-mail: ninghao.liu@uga.edu).

Ying Wang is with the College of Computer Science and Technology, Jilin University, Changchun 130012, China (e-mail: wangying2010@jlu.edu.cn).

The code is in: https://github.com/YL-wang/AdaGCL_Plus.git.

Digital Object Identifier 10.1109/TPAMI.2025.3574354

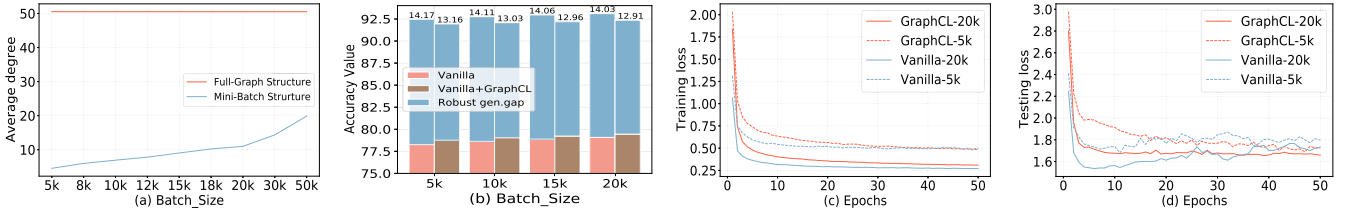


Fig. 1. (a) Topological bias under subgraph sampling, measured by the average node degree as batch size decreases. (b) Impact of topological bias on model performance: the pink and brown bars denote test accuracy under different settings; the blue bar indicates the generalization gap (train accuracy minus test accuracy). (c), (d) Training and testing loss curves under different subgraph sampling strategies.

It is non-trivial to apply the contrastive learning methods in graph batch training due to the following challenges. *First, it induces high computational and memory costs to contrast all the node pairs within a large graph required in the vanilla contrastive loss.* Some efforts have been proposed to reduce the computation cost either based on node pair sampling [22] or only measuring the single positive pair [23], [24]. However, they tend to degrade the node embeddings without sufficient contrasting pairs. *Second, the existing graph augmentation strategies are not adaptive to the sampled batch.* While the previous work transforms the whole small graph, they inevitably create noisy links among the individual subgraphs within a batch. The noisy augmented graphs prevent GNNs from learning the node embeddings informative to the downstream task [25]. *Third, a fixed perturbation rate used in graph augmentation fails to generalize across the various graph tasks.* The graph perturbation rate, i.e., the probability of adding or dropping nodes and edges, is critical to balancing similarity and diversity between the original and augmented batches, which determines the model's generalization extent in learning the invariant node embeddings. The optimal perturbation rate varies with downstream tasks and requires tedious trials to search it. There exists work on learnable data augmentation for small graphs, but their time costs are often too high with low returns [20], [26].

To address the challenges in graph batch training, we propose AdaGCL+, which introduces key innovations to improve the efficiency and robustness of graph contrastive learning. AdaGCL+ utilizes **batch-aware graph augmentation** that adapts to the structure of each batch, enhancing the similarity of positive pairs while maintaining diversity and reducing topological noise. In parallel, **batch-aware pair sampling** is combined with **subgraph-granularity contrastive loss**, which operates at the subgraph level rather than the node pair level, significantly reducing computational and memory costs. This approach improves the efficiency of contrastive learning by focusing on subgraph-level comparisons. Furthermore, we introduce an **automated augmentation strategy** based on the **Node-centric Information Bottleneck (Node-IB)** principle, which adaptively adjusts the augmentation rate during training. Node-IB helps the model focus on task-relevant information, mitigating the topological bias caused by subgraph sampling, and ensuring that node embeddings are robust to structural incompleteness. Together, these modules make AdaGCL+ more efficient and effective in learning robust node embeddings for large-scale graphs. To summarize, this work makes four key contributions:

- We provide the first comprehensive analysis of the relationship between inherent topological bias within batch sampling and the generalization robustness of Graph Neural Networks (GNNs). Through empirical observations, we demonstrate that contrastive learning can mitigate poor generalization by learning robust node embeddings that are invariant to graph structural incompleteness.
- We introduce a novel subgraph-granularity contrastive loss that utilizes batch-aware pair sampling to compare anchor node embeddings with subgraph embeddings at the batch level. This approach has a constant operational cost due to the limited cardinality of subgraphs, making it more efficient than node pairwise contrasting, thus addressing *Challenge #1* of high computational and memory costs in large graph training.
- We propose Node-IB, a key module in AdaGCL+, which reduces the topological bias inherent in subgraph sampling. Node-IB guides the model to focus on relevant task-related information while ignoring irrelevant noise, ensuring robust and generalized node embeddings. By adapting the graph augmentation process, Node-IB tackles *Challenge #2*, improving the adaptability of augmentation strategies to different sampled batches.
- Our method, AdaGCL+, introduces a batch-aware graph augmentation technique with automated data augmentation. This technique designs augmentation strategies and contrastive pair constructions depending on the batch sampling methods, addressing *Challenge #2*. It optimizes the graph perturbation rate to balance similarity and diversity among augmented graph views, improving the generalization of node embeddings. This dynamic perturbation strategy helps mitigate the problem of tuning a fixed perturbation rate, addressing *Challenge #3*.

II. RELATED WORK

Scalable graph representation learning: Existing GNNs, like GCN [6], GAT [7], and GIN [27], apply recursive message passing across neighborhoods to learn dependencies in graph data. They can usually only be trained in full batches on the small graph and are difficult to apply to the large-scale graph. In order to solve the scalability problem of GNNs with limited resources, subgraph sampling methods [13], [28], [29] and precomputing methods [1], [30], [31] have been proposed. To reduce the computation and memory overhead, the subgraph sampling methods

only provide GNNs with a small set of subgraphs, which consist of a subset of nodes and edges. Specifically, GraphSage [5], meanwhile, selects neighbors of different orders by its central node at each training, which together form a subgraph to participate in the training. Cluster-GCN [14] first performs a clustering operation on the scalable graph, divides it into clusters, and then trains the clusters in each batch. GraphSAINT [32], on its part, performs subgraph sampling on the large-scale graph at each training and trains the picked connected subgraphs. The main limitation of the subgraph-based approach is that the nodes in the training set have different features and neighbor structures than the nodes in the testing set, thus making the overfitting of the model affect its generalization ability [33], [34].

Graph contrastive learning: Recently, contrastive learning has aroused much research interest. Its key idea is to learn discriminative representations by contrasting positive and negative samples [15], [35]. Contrastive learning on graphs is an active and promising research area with broad potential applications. DGI [36] is the first GNN model utilizing the idea of contrastive learning, where the mutual information between nodes and the graph representation is defined as the contrastive metric. MVGRL [37] learns node-level and graph-level representations by performing node diffusion and comparing node embeddings with augmented graph-level summaries. Graph Barlow Twins [38] introduces a redundancy reduction objective to decorrelate embedding dimensions, while BGRL [24] adapts BYOL to graphs and removes the need for negative samples in training. Recent works have also explored the use of contrastive learning in the context of recommendation. CGI [39] proposes a learnable augmentation framework based on the information bottleneck principle, selectively preserving task-relevant links and filtering out noise in user-item interaction graphs. Light-GCL [40] avoids stochastic augmentation entirely by leveraging SVD-based graph compression to generate stable views, achieving efficient and robust contrastive learning for collaborative filtering. Although these methods show strong performance in recommendation tasks, they are often designed for bipartite graphs and assume full-graph training.

Graph data augmentation: On the basis of graph contrastive learning, graph data augmentation has also been investigated. Dropedge [23] randomly removes a portion of edges during training. Graph Contrastive Learning (GraphCL) [25] proposes five different augmentation strategies and uses a trial-and-error approach for different datasets to manually select graph augmentation and the corresponding parameters for each augmentation on different types of datasets. JOAO [21], however, adds an automatic selection module for different augmentations based on GraphCL. GCA [41] uses a set of heuristic algorithms to perform an adaptive selection of augmentation parameters. ADGCL [26], introduces an adversarial graph augmentation strategy that uses the infoMin [42] principle to avoid capturing redundant information. LG2AR [20] is an end-to-end automatic graph augmentation framework that helps encoders learn common representations at the node and graph levels by automatically learning graph augmentation strategies. However, the above data augmentation strategies are designed to use different data augmentation methods for different datasets. Nevertheless,

for different large-scale datasets, it will greatly prolong the training speed of the model, but may only bring a weak effect improvement.

III. PROBLEM STATEMENT

The undirected graph is represented by $G = (A, X)$, where $A \in \mathbb{R}^{N \times N}$ denotes adjacency matrix, $X \in \mathbb{R}^{N \times d}$ denotes feature matrix, N is a number of nodes, and d is feature vector dimension. We focus on the node classification problem: Each node v_i is associated with a feature vector $x_i \in \mathbb{R}^d$ and a label $y_i \in \mathbb{R}^C$. Given a labeled training set, the goal is to classify nodes at an unlabeled testing set by learning the mapping function of GNNs.

In this work, we adopt the batch training paradigm, one of the popular scalable approaches, to fit the mapping function of GNNs on large-scale graph data. Specifically, a batch is often represented by $\mathcal{B} = \{g_1, \dots, g_b, \dots, g_B\}$, where g_b denotes the sampled subgraph, and B is the batch size. The construction of the subgraph varies depending on the sampling approach. For example, g_b is given by a small cluster in Cluster-GCN, while it is a multi-hop subgraph centered on a training node in GraphSAGE. Under the batch training paradigm, the node embedding at the l th layer of GNNs is:

$$x_i^{(l)} = f_{\theta}^{(l)} \left(x_i^{(l-1)}, \left\{ x_j^{(l-1)} \right\}_{j \in \mathcal{N}(i) \cap \mathcal{B}} \right), \quad (1)$$

where $x_i^{(l)} \in \mathbb{R}^d$ is the embedding of node v_i learned at the l th layer. $f_{\theta}^{(l)}$ can be decomposed into message passing and update functions parameterized by weights θ , which are used to aggregate neighbor embeddings (e.g., sum, mean, or max) and generate the target node embedding (e.g., MLP), respectively. $\mathcal{N}(i)$ denotes the set of direct neighbors of node v_i . The local information of neighbors within the sampled batch is $\mathcal{N}(i) \cap \mathcal{B}$, while the out-of-batch neighbors are discarded in the message passing. We assume the number of graph convolutional layers is L , and use f_{θ} to denote the L -layer GNN encoder. The node embedding at the final layer, i.e., $h_i = f_{\theta}(x_i)$ is used for the downstream node classification task.

A. Topological Bias and Generalization Robustness Analysis

One fundamental problem of the batch training paradigm is the topological bias induced at the local subgraphs. According to (1), the neighbor information is partially provided by the local batch in the training phase. Although batch sampling gets rid of the memory explosion issue by only maintaining the node embeddings of subgraphs, it fails to present the complete neighbor set and precise topological structure. In contrast, in the testing phase, the whole graph is used for model inference. The topological bias of local subgraphs leads to imprecise embedding estimation, which deteriorates the generalization performance at the testing set.

To demonstrate the above analysis, we empirically study the relation between topological bias and generalization performance. In particular, we describe the topological bias by the node degree gap induced by batch sampling, i.e., $\Delta_{\text{deg}} =$

$\frac{1}{N} \sum_i d_i - \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} d'_i$, where d_i and d'_i is the node degrees of node v_i at the original graph and the sampled subgraphs, respectively. Following the previous efforts [34], [43], we define the generalization robustness as the model performance gap between training and testing phases, i.e., $\Delta_{\text{gen}} = p_{\text{train}} - p_{\text{test}}$, where p_{train} or p_{test} could be instantiated by node classification accuracy. We make use GraphSAINT model and the ogbn-products dataset as an example to analyze their relationship. As shown in Fig. 1(a), the node degree gap is magnified as the batch size is decreasing, since more and more neighbors are dropped in formulating the smaller batch. Meanwhile, according to Fig. 1(b), the generalization gap (i.e., blue bar) is much larger in the small batch size case. Therefore, we are able to make the following conclusion: *the more significant the topological bias of node degree gap is, the much larger the generalization performance gap will be.*

Besides, we visualize the training and testing dynamics in terms of cross entropy loss in Fig. 1(c) and (d) (i.e., blue vanilla curves). While the training loss is always minimized with stochastic gradient descent, its testing loss increases with time. Due to the topological bias, the inaccurate estimation and over-fitting at the training set results in poor generalization for the unseen testing nodes. Compared with the batch size of 5 K, the bias and over-fitting could be relieved by enhancing the batch size to 20 K to obtain a smaller testing loss. But it would incur large memory consumption.

B. Graph Contrastive Learning

Graph contrastive learning has recently attracted extensive research interest due to its robust node embedding learning by aligning the multiple transformed views of a graph. In particular, the contrastive objective aims to maximize the similarity score between the different view representations generated from the same graph. Mathematically, considering the input graph $G = (A, X)$, we use $\mathcal{T}_\alpha$ and $\mathcal{T}_\beta$ to denote graph augmentation function to obtain the two different views: $\mathcal{T}_\alpha(G) \equiv G^\alpha = (A^\alpha, X^\alpha)$ and $\mathcal{T}_\beta(G) \equiv G^\beta = (A^\beta, X^\beta)$. Given GNN encoder f_θ , we denote the node embeddings inferred from the two views by h_i^α and h_i^β , respectively. The similarity scores between view representations are often indicated by mutual information. The contrastive objective is formulated as:

$$\max_{\theta} \text{MI} \left(h_i^\alpha, h_i^\beta \right). \quad (2)$$

$\text{MI}(\cdot, \cdot)$ denotes mutual information between two embedding variables. In the context of the node classification task, considering the anchor node embedding h_i^α , the above optimization problem could be approximated by minimizing the following contrastive loss [44]:

$$\begin{aligned} & \min_{\theta} \mathcal{L}_{\text{cl}}(h_i^\alpha, h_i^\beta) \\ &= \log(1 + \exp(-\text{sim}(h_i^\alpha, h_i^\beta))) \quad [\text{Pos pair}] \\ &+ \sum_{j \neq i, j \in G^\beta} \log(1 + \exp(\text{sim}(h_i^\alpha, h_j^\beta))), \quad [\text{Neg pairs}] \end{aligned} \quad (3)$$

where $\text{sim}(\cdot, \cdot)$ is the similarity function such as the cosine similarity or a trainable MLP layers. The same nodes v_i from two augmented graphs are treated as positive pairs, whose similarity is improved to align their representations. The negative samples from view G^β are distinguished to reinforce the similarity of the positive pair. Given the order invariance of mutual information calculation, we often minimize the combinative contrastive loss $\mathcal{L}_{\text{cl}}(h_i^\alpha, h_i^\beta) + \mathcal{L}_{\text{cl}}(h_i^\beta, h_i^\alpha)$ jointly with the cross-entropy loss $\mathcal{L}_{\text{ce}}(h_i^\alpha, y_i)$ to optimize the node embedding.

The contrastive loss is a form of regularization, which ensures that the same node embeddings from two distorted graphs are consistent with each other and invariant to random distortions. This type of invariance is especially important for batch training on large-scale graph data, where the topological bias of local subgraphs can lead to imprecise embedding estimation and poor generalization performance. By aligning the node embeddings from randomly augmented batches, GNN encoder is optimized to be more robust to the missing neighbors. To investigate the relationship between contrastive learning and generalization robustness, we evaluate GraphCL, a popular contrastive learning framework for graph data [25], with GraphSAINT batch training method on ogbn-products dataset [45]. As shown in Fig. 1(b), the application of contrastive learning results in a reduced generalization performance gap compared to vanilla GraphSAINT, as well as a continuous decrease in testing loss to avoid overfitting. These promising results motivated us to further explore batch-aware contrastive learning.

C. Graph Information Bottleneck

An effective graph augmentation plays a crucial role in the generalization performance of contrastive learning algorithms. Generally, an ideal augmented graph, i.e., $\mathcal{T}(G)$, should possess the following attributes:

- *Task-relevant information retention:* Graph $\mathcal{T}(G)$ will not drop much task-relevant information. GNNs trained on the augmented graph are required to have enough knowledge to correctly classify nodes.
- *Task-irrelevant noise reduction:* Graph $\mathcal{T}(G)$ should eliminate task-irrelevant noisy nodes/edges as more as possible, which enhances the alignment between the structural shape with its specific node class.
- *Similarity-diversity trade-off:* On one hand, if the augmented graph $\mathcal{T}(G)$ looks significantly different from the original one G , the information loss will be too large to be handled by the contrastive learning to pledge node representation quality. On the other hand, if $\mathcal{T}(G)$ is close to G , it does not provide sufficient data diversity to facilitate the contrastive learning generating robust prediction in the vicinity of the original graph.

Conventional graph augmentation often fails to meet these requirements due to the random node or edge perturbation. To tackle this challenge, Graph Information Bottleneck (GIB) introduces a pivotal principle in graph representation learning: an optimal graph representation should encompass the minimum essential information required for the downstream classification task [46].

Definition 1. (Graph information bottleneck): Given a graph G and its graph-level label y , GIB trains GNN encoder f_θ to obtain the most informative compressed representation $f_\theta(G)$ by optimizing the following objective:

$$\max_{\theta} \text{MI}(f_\theta(G), y) - \mu \cdot \text{MI}(f_\theta(G), G), \quad (4)$$

where μ is a positive constant. Equation (4) explains the fundamental idea underlying GIB: after training, an optimal representation learned by GNNs should be minimally yet sufficiently informative for downstream prediction tasks. Maximizing the mutual information between representation $f_\theta(G)$ and target y demands GNNs to sufficiently learn information for downstream accurate predictions, whereas minimizing the mutual information between representation $f_\theta(G)$ and graph G restrains prediction-irrelevant graph information to be learned. It should be emphasized that GIB cannot be directly applied to vanilla GNNs, it must combine with generative models to further compute mutual information and learn robust graph representations.

The previous arts of GIB only target at graph-level classification tasks. Since our focus is the node classification problem on large-scale graphs, in this paper, we investigate how to tailor the concept of GIB to batch-aware contrastive learning. Particularly, the aforementioned requirements as well as GIB will guide the design of the graph augmentation strategy to create the desired augmented batches.

IV. ADAPTIVE SUBGRAPH CONTRASTIVE LEARNING

Despite the improvement in effectiveness, it is inefficient and unaccommodated to directly apply the contrastive learning methods to the batch training of large-scale graph data. First, as shown in (3), the contrastive loss has to compare each node with all the remaining negative samples [25], which is computationally time-consuming and memory-intractable. Second, existing designs of graph augmentation and contrastive loss are not adapted to the batch training paradigm, which could further deteriorate the downstream task performance. As defined in Section III, the local batch $\mathcal{B} = \{g_1, \dots, g_B\}$ consists of numerous individual subgraphs, where the intra/inter-subgraph connections play different structural importance, and nodes contain various classes. The nodes at a subgraph (e.g., small cluster in Cluster-GCN) tend to have dense meaningful connections and share the correlative features or labels. The existing graph augmentations based on random structural perturbation delete the informative intra-subgraph links (i.e., violating the principle of *task-relevant information-reduction*) or introduce the noisy inter-subgraph connections (i.e., violating *task-relevant noise-reduction*). Given the meaningless corrupted views, contrastive learning fails to generate accurate and robust node embeddings.

To tackle the two challenges, we propose adaptive subgraph contrastive learning (AdaGCL) to improve the generalization robustness in the batch training of large-scale graph data. In particular, we propose a subgraph-granularity contrastive loss to measure the similarities between the target node with the sampled subgraphs, instead of the large volume of node samples to save computation cost (i.e., Challenge #1). Depending on the specific batch sampling approach, we tailor the graph (or batch)

augmentation and pair sampling approaches to guarantee the informative node embedding learning (i.e., Challenge #2). The details are introduced below.

A. Subgraph-Granularity Contrastive Loss

We leverage the contrastive view between anchor node and sampled subgraphs to reduce the contrastive computation cost in large-scale graph training. Specifically, the proposed subgraph-granularity contrastive loss is:

$$\begin{aligned} \min_{\theta} \mathcal{L}_{\text{cl}}(h_i^\alpha, h_g^\beta) &= \log \left(1 + \exp(-\text{sim}(h_i^\alpha, h_g^\beta)) \right) \quad [\text{Pos pair}] \\ &+ \sum_{g' \in \text{Pos}(v_i)} \log \left(1 + \exp(-\text{sim}(h_i^\alpha, h_{g'}^\beta)) \right) \quad [\text{Pos pairs}] \\ &+ \sum_{g' \in \text{Neg}(v_i)} \log \left(1 + \exp(\text{sim}(h_i^\alpha, h_{g'}^\beta)) \right), \quad [\text{Neg pairs}] \end{aligned} \quad (5)$$

where h_g^β denotes embedding of subgraph g at β view (i.e., the augmented graph of a batch), which contains the target node v_i . We note batch that $\mathcal{B}$ is constructed by numerous sampled subgraphs, each of which is represented by $h_g^\beta = \text{READOUT}(\{h_i^\beta | v_i \in g\})$. The READOUT function can be a simple permutation invariant function (e.g., sum, max, or mean) to aggregate all the node embeddings belonging to this subgraph. $\text{Pos}(v_i)$ and $\text{Neg}(v_i)$ denote the sets of positive and negative subgraphs, respectively. We'll introduce how to sample the positive and negative sets based on the anchor label in the following section.

The above formulation has two key differences compared with the node-pairwise contrastive loss (3): (i) We measure the similarities between the anchor node and the sampled subgraphs, whose cardinality is much smaller than the nodes. Let hyperparameter K denote the sizes of $\text{Pos}(v_i)$ and $\text{Neg}(v_i)$. With a small value of K , our method is shown to achieve both superior performance and slight computation cost in the experimental section. (ii) The positive pairs are given by both the subgraph containing the anchor node as well as the subgraphs sampled at set $\text{Pos}(v_i)$, not just a single positive sample. The positive subgraph consists of nodes mainly sharing the same label with anchor node v_i . By enhancing their similarities, the model learns to cluster nodes of the same class to facilitate the downstream node classification task.

B. Batch-Aware Graph Augmentation

Based on uniform structure modification or feature masking, most of the previous efforts are developed to augment the tiny graphs (e.g., biochemical modules with tens of nodes) for the graph classification problems, or the whole small graph (e.g., citation networks with thousands of nodes) for the node classification tasks [20], [21], [25]. However, considering the batch training of large-scale graphs, the direct application of them to augment a batch will destroy the intrinsic independence among

subgraphs. As studied in experiments, such vanilla augmentation strategy often damages the node classification tasks by introducing noisy information during graph convolutions.

According to the principles in Section III-C, we propose batch-aware graph augmentation to generate new views in vicinity of a batch. The vicinity is defined to maintain the intrinsic individual subgraph structures of the batch and learn the informative node embedding for the downstream classification task. We mathematically describe the above principles with an optimization problem:

$$\begin{aligned} \min_{\theta, \mathcal{T}_\eta} \quad & \mathcal{L}_{\text{cl}}(h_i^\alpha, h_g^\beta) \\ \text{s.t.} \quad & |\cap(\mathcal{T}_\eta(g_a), \mathcal{T}_\eta(g_b))| \leq \epsilon_{\text{inter}}, \text{ for } g_a, g_b \in \mathcal{B}; \\ & |\mathcal{T}_\eta(g_a) - g_a| \leq \epsilon_{\text{intra}}, \text{ for } g_a \in \mathcal{B}. \end{aligned} \quad (6)$$

Augmentation function $\mathcal{T}_\eta(\cdot)$ is sampled from a specific distribution and co-optimized with GNNs to minimize the contrastive loss. $|\cap(\mathcal{T}_\eta(g_a), \mathcal{T}_\eta(g_b))|$ denotes the cardinality of newly generated links connecting any two individual subgraphs, which is constrained by a positive constant ϵ_{inter} . This constraint is included to avoid the overly noise between subgraphs and satisfy principle **task-irrelevant noise reduction**. $|\mathcal{T}_\eta(g_a) - g_a|$ denotes the modification scale on the subgraph, e.g., the ratio of dropping nodes or edges after applying the augmentation function. It is upper limited by ϵ_{intra} to keep the enough original knowledge and satisfy principle **task-relevant information retention**. However, the direct optimization of (6) is computationally intractable due to the large volume of nodes and the infinite candidates of augmentation functions. Although several automated contrastive learning have been proposed to optimize the augmentation function end-to-end [19], [20], [26], they can only be applied to transform the tiny biochemical molecules or the small graphs with thousands of nodes.

Instead of learning the optimal solution, we propose the adaptive augmentation function for each specific batch sampler. In particular, we focus on three widely-used batch sampling methods, namely cluster sampler, random walk sampler, and neighborhood sampler, which are used in Cluster-GCN [14], GraphSAINT [32], and GraphSAGE [5], respectively. Notably, they are adopted in many of the large-scale benchmark evaluations.¹ Correspondingly, we design the augmentation strategies of (i) cluster node dropping (CND); (ii) path feature masking (PFM); and (iii) recursive edge dropping (RED). Given the input batch $\mathcal{B}$, we treat it as α view and perturb the included individual subgraphs to generate β view. The details are introduced below.

Cluster nodes dropping: The cluster sampler first clusters the input graph into many small partitions by minimizing the edge cutting, and then constructs each batch with a fixed number of clusters (i.e., the subgraphs). Due to the sparse correlations among clusters, the adaptive augmentation strategy should preserve most of informative intra-subgraph links and avoid creating noisy inter-subgraph edges. The proposed cluster node dropping $\mathcal{T}_{\text{CND}}(g_b)$ is conducted subgraph-wisely, which removes the unimportant nodes sparsely connected within each cluster. Considering a batch where subgraph $g_b = (A_b, X_b)$ containing a number of n nodes, we randomly sample a subset

of nodes according to the normalized probability distribution of node degrees: $\mathcal{S} = \text{Sample}(d_{i1}, \dots, d_{in})$. Node $v_{im} \in g_b$ ($m \leq n$) has a higher probability to be sampled if its degree d_{im} is relatively larger. The cardinality of sampled nodes is set to $|\mathcal{S}| = n \cdot (1 - \eta)$, where η is the predefined node dropping rate. Therefore, the cluster node dropping is: $\mathcal{T}_{\text{CND}}(g_b) \equiv g_b^\beta = (A_b[\mathcal{S}, \mathcal{S}], X_b[\mathcal{S}, :])$, for each $g_b \in \mathcal{B}$.

Path feature masking: Starting from a subset of training nodes, the random walk sampler goes to certain hops for each of them and obtains the series of subgraphs to construct a batch. The chain-connecting nodes along a short path are highly similar to each other, and their features could be recovered by the neighborhood. The augmentation strategy of path feature masking works independently for each subgraph and assigns part of the nodes' features with constant parameters. Mathematically, considering subgraph $g_b = (A_b, X_b)$ at a batch, we randomly choose a subset of nodes and denote it as $\mathcal{S}$. The cardinality is set to $|\mathcal{S}| = n \cdot \eta$, where η is the masking rate. The path feature masking is then defined as: $\mathcal{T}_{\text{PFM}}(g_b) \equiv g_b^\beta = (A_b, X_b')$, where $X_b'[\mathcal{S}, :] = 0$ for each $g_b \in \mathcal{B}$.

Recursive edge dropping: Starting from a subset of training nodes, the neighborhood sampler recursively includes neighbors of different orders for each of them and formulates a batch of subgraphs. Specifically, for subgraph $g_b = (A_b, X_b)$ at a batch, the set of edges included in adjacency matrix A_b is given by $\mathcal{E} = \{\mathcal{E}_1, \dots, \mathcal{E}_L\}$, where $\mathcal{E}_1$ denotes links between the centered training node and the sampled 1-order neighbors, and $\mathcal{E}_l$ contains those connecting $(l - 1)$ -order and l -order neighbors. The edges of different orders play varying roles in learning the centered node embedding. For example, the overly dropping of $\mathcal{E}_1$ cuts off the root tubes of centered node to receive neighborhood messages, while the dropping of $\mathcal{E}_L$ has a much slighter impact. To learn the task-informative node embedding during the data augmentation, we apply the order-wise edge dropping rates: $\eta = \{\eta_1, \dots, \eta_L\}$. Edge set $\mathcal{E}_l$ is independently dropped with rate η_l , which transforms the original adjacency matrix A_b to A_b' . Formally, we define the augmentation strategy of recursive edge dropping as: $\mathcal{T}_{\text{RED}}(g_b) \equiv g_b^\beta = (A_b', X_b)$, for each $g_b \in \mathcal{B}$.

Compared with the existing augmentation strategies, there are two key differences: (i) Instead of working on the whole graph, we apply them subgraph-wisely to avoid creating noisy links between the individual subgraphs. Despite the conceptual simplicity, we pay extensive engineering efforts to modify the open sampling packages and generate the batch with explicit construction of subgraphs. (ii) Instead of using the fixed augmentation strategy, we tailor it by considering the intrinsic structure of the sampled batch. We empirically show that the contrastive learning accompanied by the unmatched augmentation function consistently worsens the downstream node classification task.

C. Batch-Aware Pair Sampling

Depending on the batch sampling techniques, we generate the positive and negative sets of subgraphs to compute the contrastive loss defined in (5). While the positive subgraph mainly consists of nodes with the same class as the anchor node, the negative subgraph contains nodes of the other classes. In

¹https://ogb.stanford.edu/docs/leader_nodeprop/

particular, we develop cluster pair sampling for cluster samplers, and root pair sampling for random walk as well as neighborhood samplers.

Pair sampling based on cluster: We target large-scale homogeneous graphs, where the nodes of similar features and patterns are often clustered together. We define the class of a subgraph (i.e., a small cluster) to be the majority class of training nodes. We then sample both $\text{Pos}(v_i)$ and $\text{Neg}(v_i)$ according to the class of anchor node v_i . Specifically, we have $|\text{Pos}(v_i)| = |\text{Neg}(v_i)| = K$.

Pair sampling based on the root: Starting from the subset of root training nodes, the random walk and neighborhood samplers generate the batch of subgraphs surrounding them. With a small walk length or neighborhood sampling order, nodes in a subgraph tend to share the same class as the root node. We thus define the class of subgraph by the root training nodes. Let v_b denote the root node of subgraph g_b . We have $\text{Pos}(v_i) = \{g_{b1}, \dots, g_{bK} | y_{v_{bk}} = y_{v_i}\}$, $\text{Neg}(v_i) = \{g_{b1}, \dots, g_{bK} | y_{v_{bk}} \neq y_{v_i}\}$.

The batch-aware pair sampling method, together with subgraph-granularity contrastive loss and batch-aware graph augmentation, enables more efficient and targeted contrastive learning by focusing on the inherent structure of the sampled subgraphs. While these techniques significantly improve the computational efficiency and robustness of node embeddings, challenges remain in automating the augmentation process to adaptively balance the similarity-diversity trade-off and further optimize the graph's structural integrity.

To address these challenges, we introduce AdaGCL+ in the next section, an extension that integrates an adaptive and automated augmentation strategy. AdaGCL+ builds upon the methods discussed in Section IV by introducing Node-centric Information Bottleneck (Node-IB) to dynamically adjust graph perturbation rates, improving the model's ability to generalize while ensuring robust, task-relevant information retention.

V. ADAGCL+: AUTOMATED GRAPH AUGMENTATION

Building upon the methods introduced in Section IV, including subgraph-granularity contrastive loss, batch-aware graph augmentation, and batch-aware pair sampling, we present AdaGCL+, a framework designed to optimize graph augmentation and contrastive learning for large-scale node classification. Central to AdaGCL+ is the Node-centric Information Bottleneck (Node-IB), which dynamically adjusts the augmentation rate (η) during training. This dynamic adjustment minimizes topological bias and ensures that task-relevant information is retained, which is especially important for large-scale node classification tasks where graph size and complexity present unique challenges.

In the context of batch-aware graph augmentation, we propose three distinct strategies—CND, PFM, and RED—each targeting two key objectives: *task-irrelevant noise reduction* and *task-relevant information-retention*. Despite these advances, a key challenge remains: automating the process to achieve the optimal **similarity-diversity trade-off**. To address this, we leverage the information bottleneck principle.

Specifically, we define the optimization objective for batch-aware graph augmentation as a node-centric information bottleneck, which encourages GNNs to focus on information related to the target label from the augmented batch while filtering out noise present in the original batch. An automation strategy based on Node-IB is then introduced, enabling the adaptive adjustment of the augmentation rate η throughout the training process.

The details of this approach are described in the following sections.

A. Automated Batch-Aware Graph Augmentation

1) *Node-Centric Information Bottleneck:* Taking the augmented graph into consideration, the objective function of GIB in (4) can be rewritten as:

$$\max_{\theta} \text{MI}(f_{\theta}(\mathcal{T}_{\eta}(G)), y) - \mu \cdot \text{MI}(f_{\theta}(\mathcal{T}_{\eta}(G)), f_{\theta}(G)), \quad (7)$$

where $\mathcal{T}_{\eta}(\cdot)$ denotes the graph augmentation function that generates new views. Instead of learning graph representations and classifying graph labels, we focus on the node-granularity prediction task. We thus propose **Node-IB** to guide the node representation learning with batch-aware graph augmentation.

Definition 2. (Node-IB): Given a batch $\mathcal{B}$ extracted from an undirected graph G , for each anchor node v with label y_v , GNN encoder f_{θ} trained on the augmented batch should obtain the most compressed yet informative node embedding, which is mathematically defined as:

$$\min_{\text{MI}_2} \max_{\text{MI}_1} \text{MI}_1(H_v^{\beta}, Y_v) + \mu \cdot \text{MI}_2(H_g^{\beta}, H_g^{\alpha}), \quad (8)$$

where $H_v^{\beta} = [h_1^{\beta}, \dots, h_{|\mathcal{B}|}^{\beta}]^T \in \mathbb{R}^{|\mathcal{B}| \times d}$, and $Y_v = [y_1, \dots, y_{|\mathcal{B}|}]^T \in \mathbb{R}^{|\mathcal{B}| \times C}$ denote the nodes' embeddings and labels, respectively, and $|\mathcal{B}|$ denotes the number of nodes included at batch $\mathcal{B}$. $H_g^{\alpha} = [h_{g1}^{\alpha}, \dots, h_{gK}^{\alpha}]$ and $H_g^{\beta} = [h_{g1}^{\beta}, \dots, h_{gK}^{\beta}]$ collect a number of K sampled subgraph embeddings within the batch. We use α to represent the original graph and β to represent the augmented graph.

The objective in (8) consists of two parts. First, $\text{MI}_1(H_v^{\beta}, Y_v)$ measures the mutual information between the node embeddings of augmented batch H_v^{β} and the node labels Y_v . By maximizing the mutual information, H_v^{β} retains the required amount of label information during the node embedding learning. Second, by minimizing mutual information $\text{MI}_2(H_g^{\beta}, H_g^{\alpha})$ between augmented batch embedding H_g^{β} and original batch embedding H_g^{α} , the augmented graph will exhibit greater diversity compared with the original graph. The complementarity in these two mutual information computations during model training enables a trade-off in similarity and diversity. However, it is inefficient to directly compute the mutual information since the hidden representations are often high-dimensional and a joint distribution of (H_v^{β}, Y_v) or $(H_g^{\beta}, H_g^{\alpha})$ has to be considered. This necessitates a surrogate objective formulation.

To address the computational challenge, we introduce two distinct objective functions in place of $\text{MI}_1(H_v^{\beta}, Y_v)$ and $\text{MI}_2(H_g^{\beta}, H_g^{\alpha})$, respectively. First, as previous research suggests, minimizing cross-entropy is tantamount to maximizing the mutual information between features and true labels [47].

Consequently, the mutual information between H_v^β and Y_v can be approximated by cross-entropy loss $\mathcal{L}_{CE}$:

$$\begin{aligned} \text{MI}_1(H_v^\beta, Y_v) &= H(H_v^\beta) - H(H_v^\beta | Y_v) \\ &\approx \mathcal{L}_{CE}(H_v^\beta, Y_v) \\ &= -\frac{1}{|B|} \sum_{i=1}^{|B|} p(y_i) \log q(y_i | h_i^\beta), \end{aligned} \quad (9)$$

where $H(\cdot)$ denotes information entropy, $p(\cdot)$ and $q(\cdot)$ are the ground truth probability distribution and the predicted probability distribution respectively. Second, to minimize the mutual information between H_g^α and H_g^β , we employ a simple surrogate method of contrastive learning. Specifically, we aim to suppress the similarity between H_g^α and H_g^β [44] and utilize the subgraph-granularity contrastive loss introduced in Section IV-A as an approximation function:

$$\text{MI}_2(H_g^\beta, H_g^\alpha) \approx \frac{1}{K} \sum_{i=1}^K [\mathcal{L}_{cl}(h_i^\alpha, h_g^\beta) + \mathcal{L}_{cl}(h_i^\beta, h_g^\alpha)]. \quad (10)$$

By combining (9) and (10), we derive the following objective to implement **Node-IB** principle in practice:

$$\begin{aligned} \text{argmin}_{\theta, \mathcal{T}_\eta} \quad & -\frac{1}{|B|} \sum_{i=1}^{|B|} p(y_i, h_i^\beta) \log q(y_i | h_i^\beta) \\ & + \frac{\mu}{K} \sum_{i=1}^K [\mathcal{L}_{cl}(h_i^\alpha, h_g^\beta) + \mathcal{L}_{cl}(h_i^\beta, h_g^\alpha)], \end{aligned} \quad (11)$$

where θ is the parameters of GNN encoder, $\mathcal{T}_\eta$ is the batch-aware graph augmentation function. According to the above optimization function, the proposed node-centric information bottleneck optimizes both the GNN encoder and augmentation strategy to learn the compressed yet informative embeddings, which are robust to the batch structure bias.

2) *Automated Similarity-Diversity Trade-Off*: Although **Node-IB** seeks an optimal trade-off regarding to the similarity and diversity between the augmented and original graphs, it lacks an automated mechanism for dynamically adjusting the generation process of the augmented graph. There have been a couple of previous works of learnable graph augmentation [20], [26]. They generally learn an importance score for each edge or node and sample the augmented graph based on the importance distribution. While these approaches show promising performances on small graphs, they are intractable to be scaled on the large-scale graphs interested in this work. Instead of laboriously tuning the importance scores, we note that augmentation rate η plays an important role in batch-aware graph augmentation. A larger value of η effectively removes most unnecessary noisy nodes or edges in subgraphs, but there's a risk of altering the informative intrinsic structures within batches. On the other hand, a smaller value of η preserves most of the batch features but limits the generation of novel views.

In order to adjust the similarity-diversity trade-off for each downstream application, we design an automated rate (AutoR)

strategy to optimize the batch-aware graph augmentation $\mathcal{T}_\eta(\cdot)$. The updating formula for augmentation rate η is defined as follows:

$$\text{AutoR} : \eta_{t+1} = \eta_t \pm \delta_t \cdot \sigma(\mathcal{L}_t - \mathcal{L}_{t-1}), \quad (12)$$

where $\mathcal{L}$ is the loss function of Node-IB in (11), t denotes training step index, $\pm$ is 'sign($\mathcal{L}_t - \mathcal{L}_{t-1}$)', $\sigma(\cdot)$ is a mapping function, and δ_t is a hyperparameter that controls the penalty intensity of loss variance on the augmentation rate. The relationship between loss variance and the augmentation rate in this formula suggests a dynamic approach to managing similarity and diversity in the training process: *High Loss Variance*: Indicates significant changes in the model's performance, prompting larger adjustments in the augmentation rate. This could lead to increased diversity in the training process, as the model explores different strategies to reduce loss. *Low Loss Variance*: Suggests stability in the model's performance, resulting in smaller adjustments to the augmentation rate. This stability could be due to the model consistently performing well (or poorly) on similar types of data, implying a focus on similarity.

Overall, this formula represents an adaptive mechanism where the augmentation rate is fine-tuned based on the recent performance trends of the model, balancing diversity and similarity in the training process.

B. Model Training

AdaGCL+ is a framework that aims at improving the generalization robustness of node classification on large-scale graphs by addressing the topological bias in batch sampling. Two innovations are made: the adaptive graph augmentation to control the generation trade-off between similarity and diversity, and the subgraph-granularity contrastive learning to reduce computation cost. Overall as shown in Fig. 2, AdaGCL+ consists of five key steps: batch sampling, batch-aware graph augmentation, embedding encoding, contrastive pair sampling, and model optimization. The detailed description of our proposed framework is given in Algorithm 1.

C. Computational Complexity Analysis

In this section, we analyze the computational complexity of AdaGCL+ for each local training batch. For clarity, we exclude the costs associated with data loading and augmentation, as these are system-dependent and vary in practice.

Let a local batch be denoted as $\mathcal{B} = \{g_1, \dots, g_B\}$, containing n nodes, each with d -dimensional feature vectors. Based on (5), the computation of the subgraph-level contrastive loss for each anchor node consists of three components. We assume that the similarity function $\text{sim}()$ is implemented as cosine similarity, and that each anchor node v_i has K positive and K negative samples (i.e., $|\text{Pos}(v_i)| = |\text{Neg}(v_i)| = K$), where $K \ll n$.

Time Complexity: Each similarity computation involves a dot product between the anchor node embedding of shape $(1 \times d)$ and a sample embedding of shape $(1 \times d)$, followed by normalization. Thus, the time complexity for computing similarity with

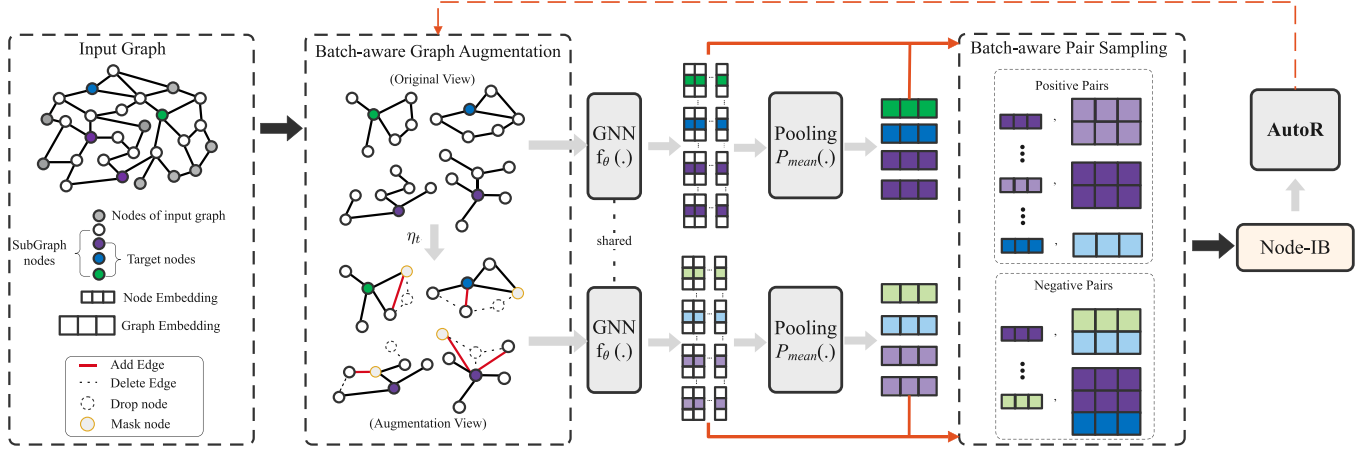


Fig. 2. Illustration of the AdaGCL+ framework. First, for each batch, the input graph is sampled to obtain SubGraphs as the original graph. We use batch-aware graph augmentation to get the augmented graph. The two views apply the GNN encoder and pooling module to obtain their node embedding and graph representation. Next, we adopt Batch-aware Pair Sampling to obtain positive and negative pairs, where the original graph provides the representation of the target node (nodes with colors) and the augmented graph provides the positive/negative representations of the subgraphs. Lastly, we train the whole model using the Node-IB method and optimize the batch-aware graph augmentation module.

Algorithm 1: Adaptive Subgraph Contrastive Learning.

Require: Large-scale Graph $G = (A, X)$, Node label $Y = [y_1^T; \dots; y_N^T]$, Batch sampler $\mathcal{Z}$, Batch-aware Graph Augmentation with initialized augmentation ratio τ_{η_1} , Encoder f_θ , Pooling $\mathcal{P}$, Batch-aware Pair Sampling $\mathcal{PS}$, The number of nodes involved in contrastive learning K , Learning rate γ_t and the penalty intensity δ_t of Node-IB;

- 1: **for** $epoch \leftarrow 1, 2, \dots, E$ **do**
- 2: **for** $Batch \leftarrow 1, 2, \dots, t, \dots, T$ **do**
- 3: Batch sampling: $\mathcal{B} = \mathcal{Z}(G)$;
- 4: Batch-aware pair sampling: $\mathcal{V} = \{v_1, \dots, v_K\} \in \mathcal{BG} = \{g_{v_1}, \dots, g_{v_K}\} \in \mathcal{BPos}(v_i), \text{Neg}(v_i) \leftarrow \mathcal{PS}(\mathcal{B}), v_i \in \mathcal{V}$; //Select K anchor nodes in the batch and construct their corresponding positive and negative contrasting subgraphs
- 5: Batch-aware Graph Augmentation : $\mathcal{B}^\alpha = \mathcal{B}, \mathcal{B}^\beta \leftarrow \tau_{\eta_t}(\mathcal{B}^\alpha); \mathcal{G}^\alpha = \mathcal{G}, \mathcal{G}^\beta \leftarrow \tau_{\eta_t}(\mathcal{G}^\alpha)$;
- 6: Embedding encoding and pooling:
 $h_i^\alpha = f_\theta(\mathcal{G}^\alpha), h_i^\beta = f_\theta(\mathcal{G}^\beta); H_v^\beta = f_\theta(\mathcal{B}^\beta);$
 $h_g^\alpha = \mathcal{P}(h_i^\alpha), h_g^\beta = \mathcal{P}(h_i^\beta)$; // h_i and h_g are the node and subgraph embeddings respectively
- 7: Calculate the Node-IB loss $\mathcal{L}_t$ by (11)
- 8: Augmentation rate update: If $t = 1$, set $\eta_{t+1} = \eta_t$; else, set $\eta_{t+1} = \eta_t + \delta_t \cdot \sigma(\mathcal{L}_t - \mathcal{L}_{t-1})$
- 9: Model optimization: $\theta_{t+1} = \theta_t - \gamma_t \cdot \nabla_{\theta} \mathcal{L}_t(\theta_t)$
- 10: **end for**
- 11: **end for**
- 12: Get the final trained network f_θ

Ensure: Conduct label prediction on the trained network.

K positives and K negatives is:

$$\mathcal{O}(2Kd^2)$$

for a single anchor node.

Summing over all n training nodes in the batch, the total time complexity becomes:

$$\mathcal{O}(2nKd^2) = \mathcal{O}(nKd^2)$$

This shows that the contrastive loss computation scales linearly with the number of nodes n and the number of positive/negative samples K , and quadratically with the feature dimension d .

Memory Complexity: For each batch, AdaGCL+ needs to store:

- Node embeddings for n training nodes: $\mathcal{O}(nd)$
- Embeddings for $2K$ contrastive samples per node: $\mathcal{O}(nKd)$
- Temporary similarity scores: $\mathcal{O}(nK)$

Thus, the total memory complexity is:

$$\mathcal{O}(nd + nKd + nK) = \mathcal{O}(nKd)$$

since $K \ll n$ and $d \gg 1$, this overhead is moderate in practice.

Empirical results provided in Section VI-B4 demonstrate that AdaGCL+ introduces only marginal runtime and memory overhead compared to the base GNN models, while providing performance improvements.

VI. EXPERIMENT RESULTS

In this section, we conduct several experiments to evaluate the effectiveness of AdaGCL+ on various large real-world datasets. The primary objective of our experiments is to answer four research questions as follows:

- **Q1:** How do different data augmentation strategies influence the generalization capabilities of subgraph sampling methods on large-scale graphs?
- **Q2:** Can the performance of batch training be effectively improved by AdaGCL+, in comparison to the vanilla model and contrastive learning baselines?

TABLE I
STATISTICS OF DATASETS (“M” STANDS FOR MULTI-CLASS CLASSIFICATION, AND “S” FOR SINGLE-CLASS)

Dataset	Nodes	Edges	Feature	Labels	Training	Validation	Testing
ogbn-products	2,449,029	61,859,140	100	47(s)	8%	2%	90%
ogbn-mag	1,939,743	21,111,007	128	349(s)	85%	9%	6%
Amazon	1,598,960	132,169,734	200	107(m)	85%	5%	10%
Yelp	716,847	6,977,410	300	100(m)	75%	10%	15%

- *Q3*: Background on batch training using subgraph sampling methods for large-scale graphs, how does each module of AdaGCL+, including batch-aware graph augmentation and pair sampling, as well as the Node-IB principle, affect its generalization performance?
- *Q4*: What is the impact of critical hyperparameters on the performance of AdaGCL+?

A. Experiment Setup

Datasets: In accordance with previous large-scale graph representation learning studies, we evaluate our approach on four benchmark datasets: Yelp [32], Amazon [32], ogbn-products [45], and ogbn-mag [48]. These datasets have node numbers of 717 K, 1598 K, 2449 K, and 19939 K, respectively. We adopt the official train/validation/test splits to evaluate our models. A more detailed statistical summary of these datasets is presented in Table I.

Backbone frameworks and baselines: We adopt a 3-layer GNNs to encode the node embeddings for all the concerned methods. To implement the batch training on large-scale graphs, we consider three widely-used backbones, namely ClusterGCN [14], GraphSAINT [32], and GraphSAGE [5]. Their sample batches are by a series of small clusters, random walks, and multi-hop neighborhoods respectively.

To evaluate the effectiveness of AdaGCL+, we compare it with GCL baseline approaches of GraphCL [25], BGRL [24], Graph Barlow Twins (G-BT) [38], Subg-Con [49], and GGD [50]. GraphCL, BGRL, G-BT, Subg-Con, and GGD are self-supervised contrastive learning approaches. We train them with the cross-entropy node classification loss. At the same time, GraphCL, BGRL, GGD, and G-BT are at the same level (node-node or graph-graph) of contrastive learning, but Subg-Con is the same as AdaGCL+ belonging to a different level (node-graph) of contrastive learning.

In addition to comparing with the above baseline based on contrastive Learning, we also did some experiments to compare data augmentation. To determine which type of data augmentation method is more suitable for batch training. Our method is compared with different data augmentation baselines of drop-edge (DE) [23], mask-node (MD) [25], and drop-node (DN) [25].

Implementations: The backbone implementations and their hyperparameter settings are provided by the publicly released repositories to pursue reliable evaluations of large-scale graphs. Due to the poor scalability of contrastive learning baselines other than Subg-Con, we only keep the training framework for these baselines. Instead, the input data is processed according to the

settings in our paper. Running on an NVIDIA TITAN RTX(24 G) GPU, all the experiments are implemented by PyTorch and the graph package of Torch Geometric.

B. Performance Analysis

1) *Generalization Improvement by Data Augmentation*: To answer question **Q1**, we compare our batch-aware graph augmentation module with classical graph data augmentations on three different batch sampling backbones. We make two key observations.

① *Only for batch training designed data augmentation can always improve model generalization performance*: In Fig 3, it is observed that the generalization performance of models trained with different subgraph sampling methods on various datasets is significantly influenced by the choice of data augmentations. However, the impact of different data augmentations can still be traced. Since the batch sampling method itself leads to reduced model generalization performance due to topological bias. A direct operation of dropping nodes or dropping edges may further amplify the topological bias. Therefore, the two data augmentations, drop-edge (DE) and drop-node(DN) as shown in Fig 3, will deplete the model performance in most cases. This phenomenon is particularly evident in GraphSAGE (the blue line). In addition, data augmentations designed for different subgraph sampling methods (ours) can boost the model’s generalization performance to some extent on different datasets.

② *The ability of contrastive learning to resolve topological bias is influenced by data augmentations*: The primary goal of data augmentation in contrastive learning is to create an augmented graph that possesses both similarity and diversity compared to the original graph. In simple terms, data augmentations should increase the variability between the two views without compromising the important information present in the original graph. While other data augmentations like DN, DE, and MN may provide some performance improvements in AdaGCL+, they are notably less effective when compared to the three data augmentations proposed in this paper, namely CND, PFM, and RED. As demonstrated in Table II, our batch-aware graph augmentation module consistently outperforms other data augmentations across all sets of experiments.

2) *Generalization Improvement by AdaGCL+*: To address research question Q2, we perform a comprehensive comparison between AdaGCL+ and baseline approaches across various batch sampling backbones and datasets. We evaluate their performance in node classification tasks and summarize the results in Table III. Based on our analysis, we have made three key observations.

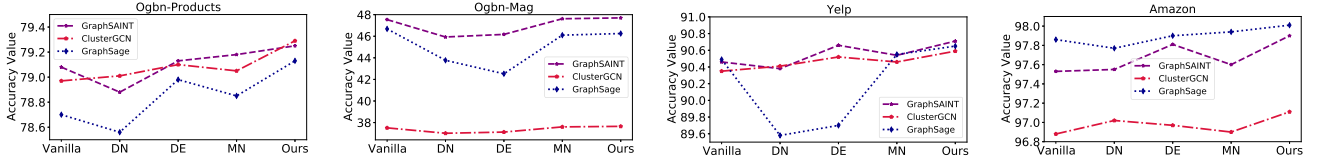


Fig. 3. Data augmentation effects on the generalization ability of models based on subgraph sampling methods on different datasets.

TABLE II
DATA AUGMENTATIONS COMBINED WITH ADAGCL+ ON THE MODEL

Backbones	Augmentation	ogbn-products	ogbn-mag	Yelp	Amazon
Cluster-GCN	Vanilla (Without Aug)	78.97 $\pm$ 0.33	37.51 $\pm$ 0.25	90.35 $\pm$ 0.09	96.88 $\pm$ 0.08
Cluster-GCN (AdaGCL+)	Drop-node (DN)	80.13 $\pm$ 0.54	37.66 $\pm$ 0.58	90.87 $\pm$ 0.14	97.37 $\pm$ 0.16
	Drop-edge (DE)	80.07 $\pm$ 0.44	37.69 $\pm$ 0.43	90.96 $\pm$ 0.18	97.63 $\pm$ 0.05
	Mask-node (MN)	79.90 $\pm$ 0.49	37.79 $\pm$ 0.49	90.52 $\pm$ 0.06	97.42 $\pm$ 0.09
	Cluster nodes dropping (CND)	80.63 $\pm$ 0.43	38.01 $\pm$ 0.43	91.23 $\pm$ 0.08	97.93 $\pm$ 0.16
GraphSAINT	Vanilla (Without Aug)	79.08 $\pm$ 0.24	47.54 $\pm$ 0.37	90.46 $\pm$ 0.12	97.53 $\pm$ 0.05
GraphSAINT (AdaGCL+)	Drop-node (DN)	79.68 $\pm$ 0.32	47.66 $\pm$ 0.44	91.01 $\pm$ 0.20	97.96 $\pm$ 0.09
	Drop-edge (DE)	79.81 $\pm$ 0.12	47.81 $\pm$ 0.38	91.21 $\pm$ 0.12	98.28 $\pm$ 0.10
	Mask-node (MN)	79.95 $\pm$ 0.19	47.88 $\pm$ 0.06	91.15 $\pm$ 0.10	98.03 $\pm$ 0.04
	Path feature masking (PFM)	80.35 $\pm$ 0.14	48.05 $\pm$ 0.33	91.48 $\pm$ 0.18	98.52 $\pm$ 0.09
GraphSAGE	Vanilla (Without Aug)	78.70 $\pm$ 0.36	46.68 $\pm$ 0.49	90.49 $\pm$ 0.15	97.86 $\pm$ 0.02
GraphSAGE (AdaGCL+)	Drop-node (DN)	79.11 $\pm$ 0.39	46.21 $\pm$ 0.58	90.88 $\pm$ 0.13	98.21 $\pm$ 0.08
	Drop-edge (DE)	79.45 $\pm$ 0.21	46.23 $\pm$ 0.46	91.19 $\pm$ 0.09	98.43 $\pm$ 0.05
	Mask-node (MN)	79.31 $\pm$ 0.22	46.54 $\pm$ 0.48	91.20 $\pm$ 0.10	98.19 $\pm$ 0.11
	Recursive edge dropping (RED)	79.88 $\pm$ 0.29	46.77 $\pm$ 0.56	91.45 $\pm$ 0.07	98.66 $\pm$ 0.14

③ *The contrastive learning methods originally proposed to encode the small graph deteriorate the node embedding learning at the local batch:* Based on the information presented in Table III, it can be observed that the contrastive learning baselines (GraphCL, BGRL, and G-BT), compared with the vanilla backbone systems, exhibit a decrease in performance ranging from -0.69% to -6.77% . In several cases, they perform even worse than the simple data augmentation of Fig. 3. This outcome can be attributed to the fact that their default augmentation strategy and contrasting pair sampling are not always adaptive to the backbone networks, leading to a detrimental effect on the significant structures within the batch. An illustrative example is the impact of GraphCL on different backbone architectures, such as GraphSAGE and Cluster-GCN, using the ogbn-products dataset. It is observed that GraphCL has a positive effect on the GraphSAGE backbone, but a negative effect on the Cluster-GCN backbone. This discrepancy can be attributed to the differences in their batch sampling techniques. Cluster-GCN aggregates nodes with similar features into clusters to form a batch, while GraphSAGE samples mixed-order neighbors of training nodes to create disconnected small subgraphs as a batch.

GraphCL tends to push node embeddings far apart from each other, which conflicts with the feature clustering characteristic of Cluster-GCN. As a result, the contrastive learning approach in GraphCL is not aligned with the clustering-based batch construction in Cluster-GCN, leading to a negative impact. However, GraphSAGE employs a diverse sampling of training node neighborhoods to construct batches. In this case, the pairwise contrasting employed by GraphCL enhances the learning

of informative nodes within the batch, resulting in a positive impact on the GraphSAGE backbone. Additionally, it is worth noting that some of the contrastive learning baselines, such as BGRL and G-BT, use only one positive pair for training, while others like GraphCL employ an uncertain number of negative pairs. These factors can lead to the degeneration of the solutions and potentially affect the overall performance of the contrastive learning baselines.

④ *AdaGCL+ generally obtains the best generalization performance on the benchmarks:* Compared with the vanilla model, AdaGCL+ achieves average improvements of 1.37%, 1.21%, and 0.89% by adapting the augmentation function and pair sampling to each backbone. The generalization robustness is compared in Fig. 4(b) and (d), where AdaGCL demonstrates the smallest performance gap (represented by the blue bar indicating the difference between training and testing accuracies). The generalization robustness is compared in Fig. 4(b) and (d), where AdaGCL+ demonstrates the smallest performance gap (represented by the blue bar indicating the difference between training and testing accuracies). This indicates that the node embeddings learned from the incomplete batch have good generalization capabilities to the original large graph with complete topology. In other words, AdaGCL+ mitigates topological bias by learning robust node embeddings that are invariant to structural dropping.

Furthermore, we provide a detailed analysis of the loss curves observed in Fig. 4(a) and (c), which depict the performance of AdaGCL+ when applied to the vanilla backbones of Cluster-GCN and GraphSAINT. In both cases, we observe that the test

TABLE III
TEST ACCURACY(%) AND ROC-AUC(%) FOR DIFFERENT MODELS ON FOUR DATASETS

Backbone sampling	Methods	ogbn-products Test Acc	ogbn-mag Test Acc	Yelp Test ROC-AUC	Amazon Test ROC-AUC	Avg. Improvement
Cluster-GCN	Vanilla	78.97 $\pm$ 0.33	37.51 $\pm$ 0.25	90.35 $\pm$ 0.09	96.88 $\pm$ 0.08	—
	GraphCL	78.15 $\pm$ 0.18	36.92 $\pm$ 0.21	89.36 $\pm$ 0.43	97.03 $\pm$ 0.08	-0.89%
	BGRL	78.43 $\pm$ 0.22	35.18 $\pm$ 0.44	<u>90.71 $\pm$ 0.11</u>	<u>97.12 $\pm$ 0.06</u>	-1.56%
	G-BT	77.51 $\pm$ 0.56	37.43 $\pm$ 0.50	87.54 $\pm$ 0.88	90.75 $\pm$ 0.45	-2.87%
	Subg-Con	<u>79.31 $\pm$ 0.37</u>	<u>37.63 $\pm$ 0.61</u>	90.56 $\pm$ 0.17	97.09 $\pm$ 0.13	0.30%
	GGD	79.01 $\pm$ 0.59	37.59 $\pm$ 0.77	90.69 $\pm$ 0.21	97.01 $\pm$ 0.04	0.16%
	AdaGCL	80.38 $\pm$ 0.40	37.83 $\pm$ 0.34	90.98 $\pm$ 0.04	97.54 $\pm$ 0.06	1.00%
	AdaGCL+	80.63 $\pm$ 0.43	38.01 $\pm$ 0.43	91.23 $\pm$ 0.08	97.93 $\pm$ 0.16	1.37%
GraphSAINT	Vanilla	79.08 $\pm$ 0.24	47.54 $\pm$ 0.37	90.46 $\pm$ 0.12	97.53 $\pm$ 0.05	—
	GraphCL	79.45 $\pm$ 0.16	47.58 $\pm$ 0.19	90.34 $\pm$ 0.06	94.24 $\pm$ 0.09	-0.74%
	BGRL	79.03 $\pm$ 0.18	46.30 $\pm$ 0.45	89.93 $\pm$ 0.23	98.02 $\pm$ 0.05	-0.69%
	G-BT	78.28 $\pm$ 0.22	47.17 $\pm$ 0.38	88.25 $\pm$ 0.46	91.61 $\pm$ 0.58	-2.57%
	Subg-Con	79.38 $\pm$ 0.11	47.55 $\pm$ 0.36	90.89 $\pm$ 0.09	98.17 $\pm$ 0.07	0.41%
	GGD	<u>79.61 $\pm$ 0.31</u>	<u>47.72 $\pm$ 0.49</u>	<u>90.92 $\pm$ 0.16</u>	<u>98.29 $\pm$ 0.12</u>	0.59%
	AdaGCL	80.19 $\pm$ 0.16	47.91 $\pm$ 0.21	91.17 $\pm$ 0.05	98.41 $\pm$ 0.04	0.97%
	AdaGCL+	80.35 $\pm$ 0.14	48.05 $\pm$ 0.33	91.48 $\pm$ 0.18	98.52 $\pm$ 0.09	1.21%
GraphSAGE	Vanilla	78.70 $\pm$ 0.36	46.68 $\pm$ 0.49	90.49 $\pm$ 0.15	97.86 $\pm$ 0.02	—
	GraphCL	<u>79.13 $\pm$ 0.27</u>	42.14 $\pm$ 0.33	87.12 $\pm$ 0.67	95.61 $\pm$ 0.11	-3.80%
	BGRL	79.02 $\pm$ 0.16	42.77 $\pm$ 0.40	90.91 $\pm$ 0.08	98.16 $\pm$ 0.03	-1.79%
	G-BT	77.17 $\pm$ 0.40	40.20 $\pm$ 0.58	85.38 $\pm$ 0.86	92.34 $\pm$ 0.32	-6.77%
	Subg-Con	78.89 $\pm$ 0.33	44.35 $\pm$ 0.50	<u>91.07 $\pm$ 0.12</u>	98.11 $\pm$ 0.16	-0.96%
	GGD	79.21 $\pm$ 0.43	45.13 $\pm$ 0.61	90.88 $\pm$ 0.05	<u>98.23 $\pm$ 0.18</u>	-0.46%
	AdaGCL	79.61 $\pm$ 0.17	46.27 $\pm$ 0.35	91.24 $\pm$ 0.12	98.49 $\pm$ 0.03	0.54%
	AdaGCL+	79.88 $\pm$ 0.29	46.77 $\pm$ 0.56	91.45 $\pm$ 0.07	98.66 $\pm$ 0.14	0.89%

The best case is in bold and expresses the next best results in the underline. Avg. Improvement in the last column means the average improvement of AdaGCL- and baselines over the vanilla model. We run 10 times and report the mean $\pm$ standard deviation.

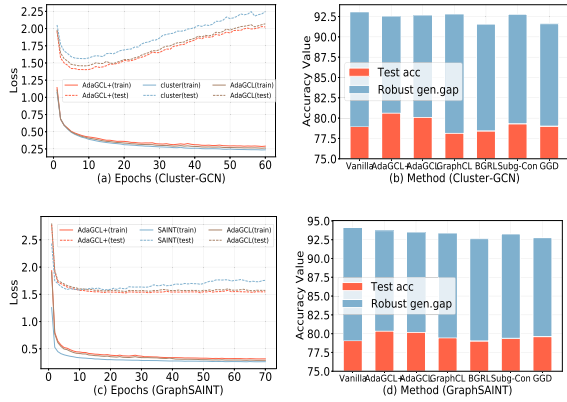


Fig. 4. Loss curves and test accuracy of Cluster-GCN and GraphSAINT on ogbn-products.

loss curves, indicated by the tomato dotted lines, converge to optimal points with significantly smaller loss values compared to the vanilla models (i.e., blue dotted lines). This demonstrates that AdaGCL+ effectively guides the learning process toward better solutions that improve the performance of the test data. It is worth noting that the training loss of AdaGCL+ (i.e., tomato lines) is slightly higher than that of the vanilla models (i.e., blue lines), indicating a regularization effect introduced by the contrasting

pairs. This regularization prevents the model from overfitting to the training set, ensuring that the learned node embeddings generalize well to unseen testing data. The smaller gap between the training and testing loss curves in AdaGCL+ suggests that it achieves a better trade-off between fitting the training data and generalizing to new, unseen instances. By incorporating the contrasting pairs into the training process, AdaGCL+ encourages the model to capture essential structures and features while avoiding the memorization of training samples. As a result, AdaGCL+ achieves higher generalisation performance on an unseen test set.

3) *Ablation Studies*: To answer question Q3, we perform ablation experiments in Table IV to analyze the individual components of AdaGCL+. Specifically, we investigate the impact of each element, such as Node-IB (Node-Centric Information Bottleneck) and AutoR (the automated rate of Node-IB guidance).

5) *Optimization strategy based on Node-Centric Information Bottleneck plays a crucial role in model performance*: In these ablation experiments, we compare the performance of AdaGCL+ with different combinations of its elements. By systematically removing or modifying specific components, we can assess their contributions to the overall performance of AdaGCL+ and understand their role in enhancing the model's capabilities. First, we introduce the original AdaGCL [51] (i.e., w/o Node-IB, AutoR) which has been upgraded compared to

TABLE IV
ABLATION STUDY OF CRITICAL COMPONENTS

Backbones	Variants	ogbn-products	ogbn-mag	Yelp	Amazon
Cluster-GCN	Vanilla	78.97 $\pm$ 0.33	37.51 $\pm$ 0.25	90.35 $\pm$ 0.09	96.88 $\pm$ 0.08
	AdaGCL	80.31 $\pm$ 0.40	37.83 $\pm$ 0.34	90.98 $\pm$ 0.04	97.54 $\pm$ 0.06
	w/ AutoR	80.28 $\pm$ 0.68	37.88 $\pm$ 0.53	91.01 $\pm$ 0.12	97.62 $\pm$ 0.21
	w/ Node-IB	80.45 $\pm$ 0.40	37.85 $\pm$ 0.21	91.10 $\pm$ 0.09	97.80 $\pm$ 0.14
	AdaGCL+	80.63 $\pm$ 0.43	38.01 $\pm$ 0.43	91.23 $\pm$ 0.08	97.93 $\pm$ 0.16
GraphSAINT	Vanilla	79.08 $\pm$ 0.24	47.54 $\pm$ 0.37	90.46 $\pm$ 0.12	97.53 $\pm$ 0.05
	AdaGCL	80.19 $\pm$ 0.16	47.91 $\pm$ 0.21	91.17 $\pm$ 0.05	98.41 $\pm$ 0.04
	w/ AutoR	80.23 $\pm$ 0.24	47.86 $\pm$ 0.20	91.18 $\pm$ 0.20	98.43 $\pm$ 0.21
	w/ Node-IB	80.28 $\pm$ 0.12	47.94 $\pm$ 0.29	91.43 $\pm$ 0.10	98.48 $\pm$ 0.06
	AdaGCL+	80.35 $\pm$ 0.14	48.05 $\pm$ 0.33	91.48 $\pm$ 0.18	98.52 $\pm$ 0.09
GraphSAGE	Vanilla	78.70 $\pm$ 0.36	46.68 $\pm$ 0.49	90.49 $\pm$ 0.15	97.86 $\pm$ 0.02
	AdaGCL	79.61 $\pm$ 0.17	46.27 $\pm$ 0.35	91.24 $\pm$ 0.12	98.49 $\pm$ 0.03
	w/ AutoR	79.58 $\pm$ 0.34	46.30 $\pm$ 0.72	91.28 $\pm$ 0.21	98.46 $\pm$ 0.18
	w/ Node-IB	79.69 $\pm$ 0.21	46.45 $\pm$ 0.32	91.35 $\pm$ 0.12	98.54 $\pm$ 0.12
	AdaGCL+	79.88 $\pm$ 0.29	46.77 $\pm$ 0.56	91.45 $\pm$ 0.07	98.66 $\pm$ 0.14

TABLE V
COMPARISON OF TRAINING TIME/(M) AND MEMORY USAGE ACROSS
DIFFERENT METHODS ON GRAPHSAINT

Methods	ogbn-products		Yelp		ogbn-mag		Amazon	
	Time	Mem	Time	Mem	Time	Mem	Time	Mem
Vanilla	8.26m	5.86G	4.82m	7.28G	4.37m	4.83G	13.25m	5.29G
GraphCL	30.37m	10.95G	34.25m	23.24G	36.49m	18.08G	45.32m	15.35G
BGRL	22.14m	9.17G	24.56m	9.73G	19.21m	8.16G	35.38m	6.93G
G-BT	25.83m	9.41G	31.24m	7.91G	24.29m	7.52G	54.47m	6.50G
Subg-Con	24.75m	7.38G	28.92m	7.69G	22.03m	6.23G	48.61m	6.79G
GGD	20.09m	6.49G	23.21m	7.59G	18.37m	5.19G	36.36m	5.66G
AdaGCL	21.83m	7.52G	25.76m	7.73G	21.49m	6.07G	37.32m	6.28G
AdaGCL+	22.21m	7.64G	27.44m	7.81G	22.56m	6.15G	39.64m	6.40G

TABLE VI
THE INFLUENCE OF AUGMENTATION PARAMETER η

Dataset	η	GraphSAINT	Cluster-GCN
ogbn-products	0.10	80.01	80.11
	0.15	80.09	80.15
	0.20	80.23	80.26
	0.25	80.21	80.31
	0.30	80.13	80.44
	AutoR	80.35	80.63

Vanilla. With the introduction of data augmentation and contrastive learning, the model's generalization performance is greatly improved. After that, we add AutoR to AdaGCL, which is still tuned by Node-IB. However, unlike AdaGCL+, its model optimization is not done by $\mathcal{L}$. Therefore, AutoR is not always effective in improving AdaGCL. In contrast, when the model is optimized based on Node-IB (i.e., w/ Node-IB), its generalization performance will be improved. This is enough to prove the rationality and importance of Node-IB. When using both Node-IB and AutoR (i.e., AdaGCL+), a greater improvement to AdaGCL is achieved by optimizing the model using Node-IB and adjusting the data augmentation rate η with each batch to obtain a better augmented graph.

The results of the ablation experiments shed light on the importance of each element and provide insights into how they contribute to the effectiveness of AdaGCL+. This analysis helps us gain a deeper understanding of the mechanisms behind AdaGCL+'s performance and provides valuable guidance for further improvements and optimizations.

4) *Computational Complexity Comparison*: Table V compares training time per batch and memory cost between AdaGCL+ and other methods. The final results presented in the table may not be the same as the analysis in Section IV-E. The reason for this is that each baseline contains different dimensional projection layers, which increases the number of parameters and computational effort. Overall, our method has a comparable or even less memory footprint than other baselines. At the same time, the run time is close to the other contrastive approaches, and slightly higher than the vanilla backbone due

to the contrastive loss computation. Our method achieves higher performance without many hardware costs and with efficiency. Furthermore, compared to AdaGCL, AdaGCL+ grows only partially in memory loss but has a higher performance improvement.

5) *Hyperparameter Studies*: To answer the research question Q4, we conduct experiments on some hyperparameters introduced by AdaGCL+ in the training process.

⑥ *Augmentation rate affects the performance of contrastive learning*: As shown in Table VI, η controls the rate of data augmentation. An oversized η leads to a significant structural difference between the augmented graph and the original graph. Consequently, some useful information cannot be recovered during the training process, affecting the performance of contrastive learning. An excessively small η makes the information from the augmented graph and the original graph quite similar. They do not provide sufficient structural perturbation information to learn the robust node embeddings. Therefore, it can be concluded that the augmented graph needs to be as different as possible from the original graph, while still retaining enough information from the original graph. The η is controlled by the Perturbation rate of Node-IB guidance (AutoR), which adjusts with the change of the training batch to find the better-augmented graph in each batch. In addition, due to the convergence property of Node-IB, the perturbation rate η of data augmentation will eventually be in a flat state, as shown in Fig. 5.

In general, there is an increase in the perturbation rate η during the early stages of training. This prompts the GNN encoder f_θ to rapidly learn diverse information between the

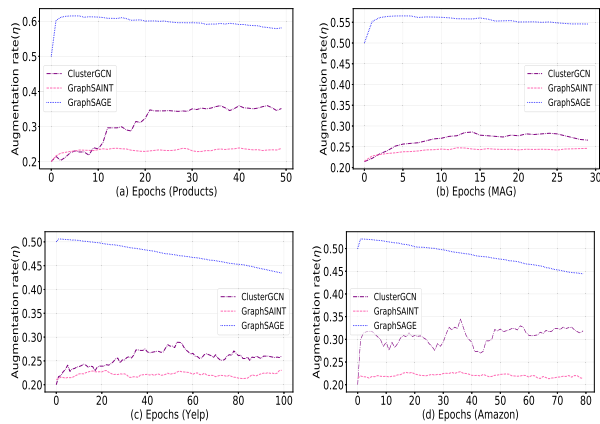


Fig. 5. The variation of perturbation rate η of data augmentation on different backbones and datasets.

augmented and original graphs. As the number of training steps increases, $\mathcal{L}$ of Node-IB will converge and $(\mathcal{L}_t - \mathcal{L}_{t-1})$ close to 0. Consequently, η eventually stabilizes at a constant value. At this point, η plays a crucial role in balancing the similarity and diversity between the augmented graphs and original graphs, maximizing the effectiveness of contrastive learning to improve the model's generalization performance.

VII. CONCLUSION

In this paper, we highlight an important issue in large-scale graph training, which is the presence of topological bias in batch sampling methods due to the limited number of neighbors available in a local batch compared to the full graph structure used in model inference. To address this research gap and mitigate the topological bias, we propose a novel approach called Adaptive Subgraph Contrastive Learning (AdaGCL+). By incorporating adaptive subgraph contrastive learning, batch-aware graph augmentation, and the node-centric information bottleneck with automated rate, AdaGCL+ demonstrates superior scalability, stability, and performance improvements compared to existing methods. It provides a promising approach for large-scale subgraph-based contrastive learning and opens avenues for further advancements in the field.

REFERENCES

- [1] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.
- [2] K. Zhou et al., "Dirichlet energy constrained learning for deep graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 21834–21846.
- [3] K. Zhou, X. Huang, Y. Li, D. Zha, R. Chen, and X. Hu, "Towards deeper graph neural networks with differentiable group normalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4917–4928.
- [4] T. Chen et al., "Bag of tricks for training deeper graph neural networks: A comprehensive benchmark study," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2769–2781, Mar. 2023.
- [5] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [7] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv: 1710.10903*.
- [8] M. Sun, K. Zhou, X. He, Y. Wang, and X. Wang, "GPPT: Graph pre-training and prompt tuning to generalize graph neural networks," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 1717–1727.
- [9] K. Guo, K. Zhou, X. Hu, Y. Li, Y. Chang, and X. Wang, "Orthogonal graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 3996–4004.
- [10] Z. Liu, K. Zhou, F. Yang, L. Li, R. Chen, and X. Hu, "EXACT: Scalable graph neural networks training via extreme activation compression," in *Proc. 10th Int. Conf. Learn. Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=vkaMaq95_rX
- [11] Y. Wang, H. Xue, and X. Wang, "A two-stage co-adversarial perturbation to mitigate out-of-distribution generalization of large-scale graph," *Expert Syst. Appl.*, vol. 255, 2024, Art. no. 124472.
- [12] J. Chen, T. Ma, and C. Xiao, "FastGCN: Fast learning with graph convolutional networks via importance sampling," 2018, *arXiv: 1801.10247*.
- [13] J. Chen, J. Zhu, and L. Song, "Stochastic training of graph convolutional networks with variance reduction," 2017, *arXiv: 1710.10568*.
- [14] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "ClusterGCN: An efficient algorithm for training deep and large graph convolutional networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 257–266.
- [15] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, "InfoGraph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization," 2019, *arXiv: 1908.01000*.
- [16] J.-B. Grill et al., "Bootstrap your own latent—a new approach to self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21271–21284.
- [17] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, "Barlow twins: Self-supervised learning via redundancy reduction," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12310–12320.
- [18] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [19] D. Xu, W. Cheng, D. Luo, H. Chen, and X. Zhang, "InfoGCL: Information-aware graph contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 30414–30425.
- [20] K. Hassani and A. H. Khasahmadi, "Learning graph augmentations to learn graph representations," 2022, *arXiv:2201.09830*.
- [21] Y. You, T. Chen, Y. Shen, and Z. Wang, "Graph contrastive learning automated," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12121–12132.
- [22] H. Zhao, X. Yang, Z. Wang, E. Yang, and C. Deng, "Graph debiased contrastive learning with joint representation clustering," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 3434–3440.
- [23] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," 2019, *arXiv: 1907.10903*.
- [24] S. Thakoor, C. Tallec, M. G. Azar, R. Munos, P. Veličković, and M. Valko, "Bootstrapped representation learning on graphs," 2021, *arXiv:2102.06514*.
- [25] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5812–5823.
- [26] S. Suresh, P. Li, C. Hao, and J. Neville, "Adversarial graph augmentation to improve graph contrastive learning," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 15920–15933, 2021.
- [27] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. 7th Int. Conf. Learn. Representations New Orleans, LA, USA*, 2019, pp. 6–9. [Online]. Available: <https://openreview.net/forum?id=ryGs6iA5Km>
- [28] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 974–983.
- [29] H. Gao, Z. Wang, and S. Ji, "Large-scale learnable graph convolutional networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1416–1424.
- [30] E. Rossi, F. Frasca, B. Chamberlain, D. Eynard, M. Bronstein, and F. Monti, "Sign: Scalable inception graph neural networks," 2020, *arXiv: 2004.11198*.
- [31] A. Bojchevski et al., "Scaling graph neural networks with approximate pagerank," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 2464–2473.

- [32] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "GraphSAINT: Graph sampling based inductive learning method," 2019, *arXiv:1907.04931*.
- [33] K. Zhou et al., "Adaptive label smoothing to regularize large-scale graph training," 2021, *arXiv:2108.13555*.
- [34] H. Xue et al., "Cap: Co-adversarial perturbation on weights and features for improving generalization of graph neural networks," 2021, *arXiv:2110.14855*.
- [35] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1150–1160.
- [36] P. Velickovic, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *Proc. Int. Conf. Learn. Representations (Poster)*, 2019, Art. no. 4.
- [37] K. Hassani and A. H. Khasahmadi, "Contrastive multi-view representation learning on graphs," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 4116–4126.
- [38] P. Bielak, T. Kajdanowicz, and N. V. Chawla, "Graph barlow twins: A self-supervised representation learning framework for graphs," 2021, *arXiv:2106.02466*.
- [39] C. Wei, J. Liang, D. Liu, and F. Wang, "Contrastive graph structure learning via information bottleneck for recommendation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 20407–20420.
- [40] X. Cai, C. Huang, L. Xia, and X. Ren, "LightGCL: Simple yet effective graph contrastive learning for recommendation," in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023, pp. 1–5. [Online]. Available: <https://openreview.net/forum?id=FKXVK9dyMM>
- [41] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph contrastive learning with adaptive augmentation," in *Proc. Web Conf.*, 2021, pp. 2069–2080.
- [42] Y. Tian, C. Sun, B. Poole, D. Krishnan, C. Schmid, and P. Isola, "What makes for good views for contrastive learning?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 6827–6839.
- [43] D. Wu, S.-T. Xia, and Y. Wang, "Adversarial weight perturbation helps robust generalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 2958–2969.
- [44] Y. Liu et al., "Graph self-supervised learning: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 5879–5900, Jun. 2022.
- [45] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 22118–22133.
- [46] T. Wu, H. Ren, P. Li, and J. Leskovec, "Graph information bottleneck," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 20437–20448.
- [47] M. Boudiaf et al., "A unifying mutual information view of metric learning: Cross-entropy vs. pairwise losses," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 548–564.
- [48] K. Wang, Z. Shen, C. Huang, C.-H. Wu, Y. Dong, and A. Kanakia, "Microsoft academic graph: When experts are not enough," *Quantitative Sci. Stud.*, vol. 1, no. 1, pp. 396–413, 2020.
- [49] Y. Jiao, Y. Xiong, J. Zhang, Y. Zhang, T. Zhang, and Y. Zhu, "Sub-graph contrast for scalable self-supervised graph representation learning," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 222–231.
- [50] Y. Zheng, S. Pan, V. Lee, Y. Zheng, and P. S. Yu, "Rethinking and scaling up graph contrastive learning: An extremely efficient approach with group discrimination," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 10809–10820.
- [51] Y. Wang, K. Zhou, R. Miao, N. Liu, and X. Wang, "AdaGCL: Adaptive subgraph contrastive learning to generalize large-scale graph training," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 2046–2055.



Yili Wang received the master's degree from the School of Software, Jilin University. He is currently working toward the PhD degree with the School of Artificial Intelligence, Jilin University, supervised by associate professor Xin Wang. His research focuses on large-scale graph representation learning, including node classification, molecular graph classification, graph OOD detection, and generalization. He specializes in techniques, such as contrastive learning, sharpness-aware minimization, adversarial learning, and diffusion models.



Yaohua Liu received the BE degree in automation from the South China University of Technology, Guangzhou, China, in 2015, and the PhD degree in control theory and control engineering from the University of Science and Technology of China, Hefei, China, in 2020. Since August 2020, she has been with the School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing, China. Her current research interests include decentralized network optimization and its applications.



Ninghao Liu (Member, IEEE) is an assistant professor with the University of Georgia. His research interests are explainable AI (XAI), graph mining, model fairness, and outlier detection. He has published refereed papers at recognized venues, such as KDD, WWW, CIKM, ICML, ICLR, NeurIPS, and WSDM, etc. His work won the Outstanding Paper Award, in ICML 2022, the Best Paper Award Shortlist, in WWW 2019, and the Best Paper Award Candidate, in ICDM 2019.



Rui Miao received the master's degree from the School of Artificial Intelligence, Jilin University. He is currently working toward the PhD degree with the School of Artificial Intelligence, Jilin University, supervised by associate professor Xin Wang. His research focuses on large-scale graph representation learning, including node classification, link prediction, and robustness of graphs. He specializes in techniques such as graph representation learning, self-supervised learning, and trustworthy machine learning.



Ying Wang received the PhD degree in School of Mathematics from Jilin University, in 2015. She is a professor of College of Computer Science and Technology with Jilin University. From 2013 to 2014, she was a visiting research scholar with Arizona State University (ASU). Her research interests include machine learning, social networks and data mining. She has published articles in top-tier conferences, such as WWW, AAAI, IJCAI, etc.



Xin Wang is an associate professor in the School of Artificial Intelligence, Jilin University, China. His research interests include trustworthy graph neural networks, out-of-distribution generalization of graph data, knowledge editing, and fairness in large language models. Dr. Wang is a CCF Outstanding Member and a distinguished talent in Jilin Province. He serves as an executive committee member of the CCF Computer Applications Technical Committee, as well as a member of the Big Data and Artificial Intelligence and Pattern Recognition Technical Committees. He is also a member of the Social Media Processing Technical Committee of the Chinese Information Society. Dr. Wang has led multiple national projects, including NSFC funding, and has published more than 20 papers in top AI conferences and journals, such as ICML, ICLR, KDD, and ACL. He serves as area chair for ACL and Senior Program Committee member for AAAI.